



Design and Implementation

By
Dr. Muhammad Bilal

Design and implementation



- ✧ Software design and implementation is the stage in the software engineering process at which an executable software system is developed.
- ✧ Software design and implementation activities are invariably inter-leaved.
 - Software design is a creative activity in which you identify software components and their relationships, based on a customer's requirements.
 - Implementation is the process of realizing the design as a program.

Software Analysis & Design Tools



- ✧ Software analysis and design includes all activities, which help the transformation of requirement specification into implementation.
- ✧ Software analysis and design is the intermediate stage, which helps human-readable requirements to be transformed into actual code.
- ✧ Let us see few analysis and design tools used by software designers:
 - Data Flow Diagram

Data Flow Diagram



- ✧ Data flow diagram is graphical representation of flow of data in an information system.
- ✧ It is capable of depicting incoming data flow, outgoing data flow and stored data.
- ✧ DFD can represent Source, destination, storage and flow of data using the following set of components



Data Flow Diagram

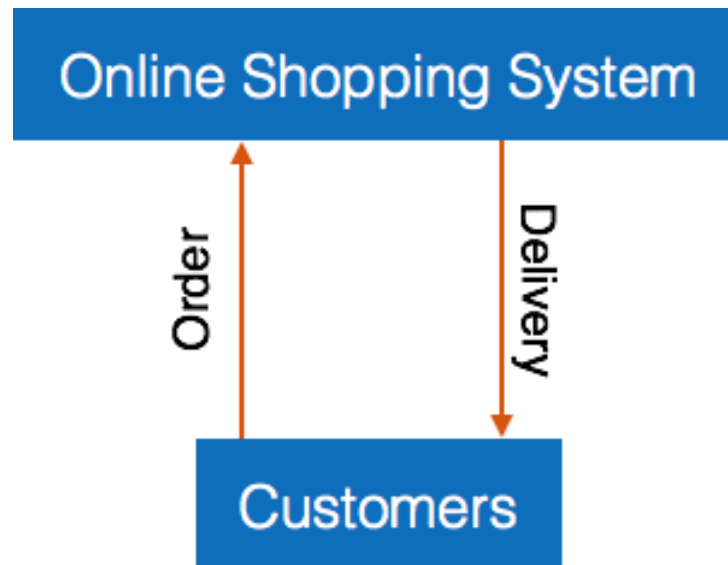


- ✧ **Entities** - Entities are source and destination of information data. Entities are represented by a rectangles with their respective names.
- ✧ **Process** - Activities and action taken on the data are represented by Circle or Round-edged rectangles.
- ✧ **Data Storage** - There are two variants of data storage - it can either be represented as a rectangle with absence of both smaller sides or as an open-sided rectangle with only one side missing.
- ✧ **Data Flow** - Movement of data is shown by pointed arrows. Data movement is shown from the base of arrow as its source towards head of the arrow as destination.

Levels of DFD



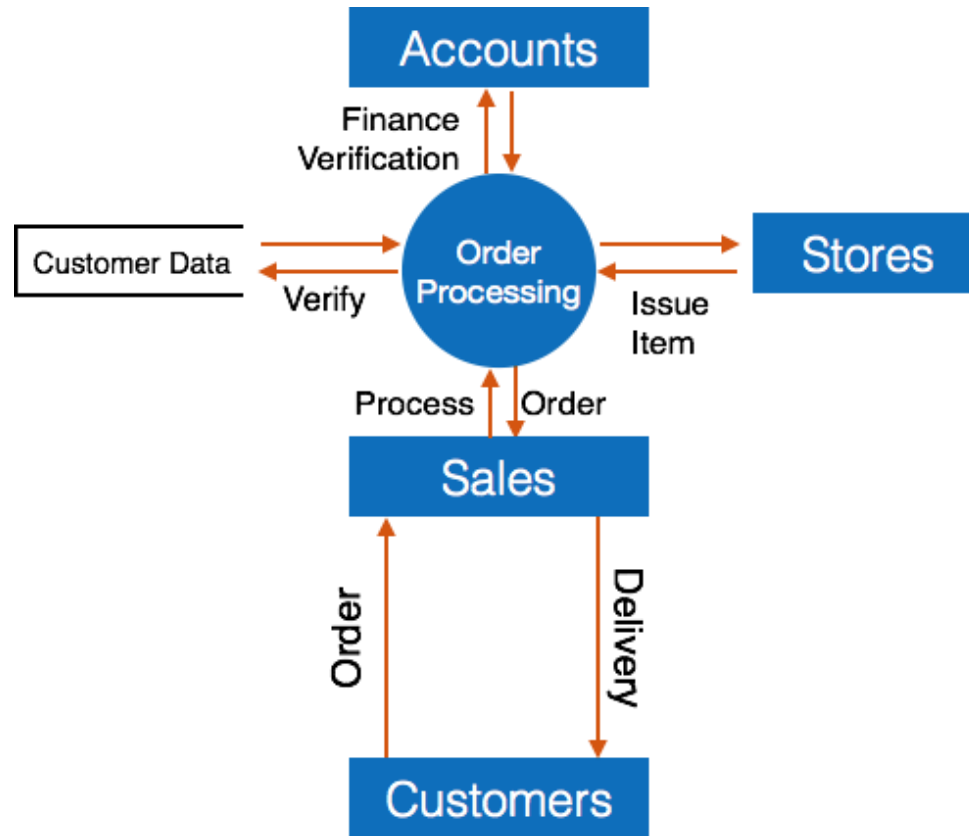
- ✧ **Level 0** - Highest abstraction level DFD is known as Level 0 DFD, which depicts the entire information system as one diagram concealing all the underlying details. Level 0 DFDs are also known as context level DFDs.



Levels of DFD



- ✧ **Level 1** - The Level 0 DFD is broken down into more specific, Level 1 DFD.



Levels of DFD



- ✧ **Level 2** - At this level, DFD shows how data flows inside the modules mentioned in Level 1.

Structure Charts

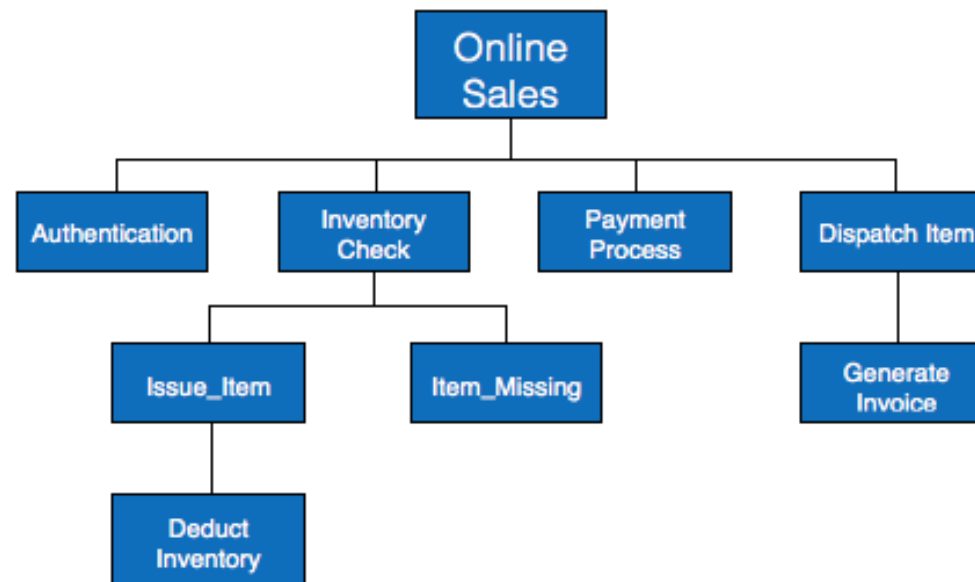


- ✧ Structure chart is a chart derived from Data Flow Diagram.
- ✧ It represents the system in more detail than DFD.
- ✧ It breaks down the entire system into lowest functional modules, describes functions and sub-functions of each module of the system to a greater detail than DFD.

HIPO Diagram



- ✧ HIPO (Hierarchical Input Process Output) diagram is a combination of two organized method to analyze the system and provide the means of documentation
- ✧ Their graphical representation makes it easier for designers and managers to get the pictorial idea of the system structure.



Structured English



- ✧ Most programmers are unaware of the large picture of software so they only rely on what their managers tell them to do.
- ✧ It is the responsibility of higher software management to provide accurate information to the programmers to develop accurate yet fast code. E.g. Pseudo-Code
- ✧

Example



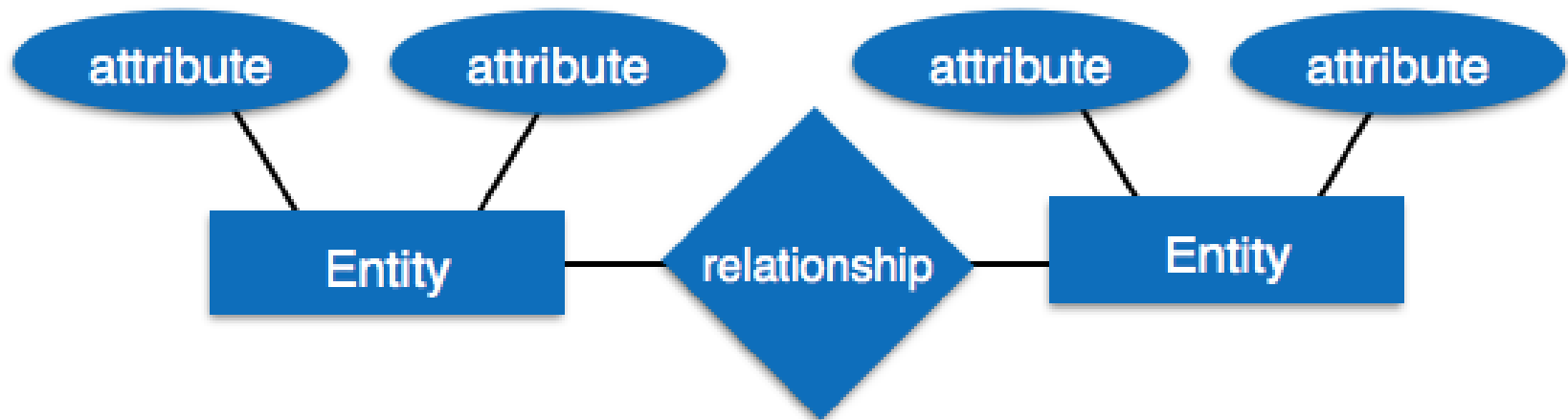
Program to print Fibonacci up to n numbers

```
void function Fibonacci
Get value of n;
Set value of a to 1;
Set value of b to 1;
Initialize i to 0
for (i=0; i< n; i++)
{
    if a greater than b
    {
        Increase b by a;
        Print b;
    }
    else if b greater than a
    {
        increase a by b;
        print a;
    }
}
```

Entity-Relationship Model



- ✧ Entity-Relationship model is a type of database model based on the notion of real world entities and relationship among them.
- ✧ We can map real world scenario onto ER database model.
- ✧ ER Model creates a set of entities with their attributes, a set of constraints and relation among them.



Entity-Relationship Model



- ✧ **Entity** - An entity in ER Model is a real world being, which has some properties called ***attributes***.
 - Every attribute is defined by its corresponding set of values, called ***domain***.
 - For example, Consider a school database. Here, a student is an entity. Student has various attributes like name, id, age and class etc.
- ✧ **Relationship** - The logical association among entities is called ***relationship***.
 - Relationships are mapped with entities in various ways.
 - Mapping cardinalities define the number of associations between two entities.

Software Design Complexity



- ✧ The term complexity stands for state of events or things, which have multiple interconnected links and highly complicated structures.
- ✧ In software programming, as the design of software is realized, the number of elements and their interconnections gradually emerge to be huge, which becomes too difficult to understand at once.
- ✧ Software design complexity is difficult to assess without using complexity metrics and measures.

Cyclomatic Complexity Measures



- ✧ Every program encompasses statements to execute in order to perform some task and other decision-making statements that decide, what statements need to be executed.
- ✧ These decision-making constructs change the flow of the program.
- ✧ It is graph driven model that is based on decision-making constructs of program such as if-else, do-while, repeat-until, switch-case and goto statements.

Cyclomatic Complexity Measures



- ✧ To calculate Cyclomatic complexity of a program module, we use the formula –

$$V(G) = e - n + 2$$

- Where
- e is total number of edges
- n is total number of nodes

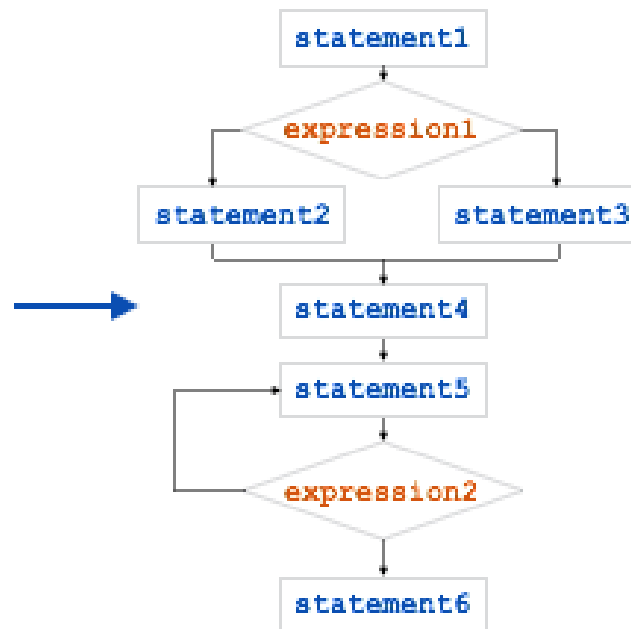
Example



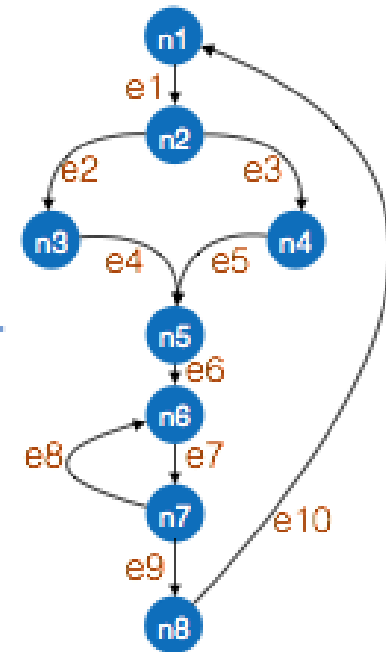
Code

```
statement1  
If expression1  
    statement2  
else  
    statement3  
statement4  
do  
    statement5  
while expression2  
statement6
```

Flow-Chart



Flow-Graph



$e = 10$

$n = 8$

$\text{Cyclomatic Complexity} = 10 - 8 + 2 = 4$

Programming style



- ✧ Programming style is set of coding rules followed by all the programmers to write the code.
- ✧ When multiple programmers work on the same software project, they frequently need to work with the program code written by some other developer.
- ✧ This becomes tedious or at times impossible, if all developers do not follow some standard programming style to code the program.

Programming style



- ✧ An appropriate programming style includes using function and variable names relevant to the intended task, using well-placed indentation, commenting code for the convenience of reader and overall presentation of code.
- ✧ This makes the program code readable and understandable by all, which in turn makes debugging and error solving easier.
- ✧ Also, proper coding style helps ease the documentation and updation.

Coding Guidelines



- ✧ Practice of coding style varies with organizations, operating systems and language of coding itself.
 - **Naming conventions** - This section defines how to name functions, variables, constants and global variables.
 - **Indenting** - This is the space left at the beginning of line, usually 2-8 whitespace or single tab.
 - **Whitespace** - It is generally omitted at the end of line.
 - **Operators** - Defines the rules of writing mathematical, assignment and logical operators. For example, assignment operator '=' should have space before and after it, as in "x = 2".
 - **Control Structures** - The rules of writing if-then-else, case-switch, while-until and for control flow statements solely and in nested fashion.

Coding Guidelines



- **Line length and wrapping** - Defines how many characters should be there in one line, mostly a line is 80 characters long. Wrapping defines how a line should be wrapped, if it is too long.
- **Functions** - This defines how functions should be declared and invoked, with and without parameters.
- **Variables** - This mentions how variables of different data types are declared and defined.
- **Comments** - This is one of the important coding components, as the comments included in the code describe what the code actually does and all other associated descriptions. This section also helps creating help documentations for other developers.

Software User Interface Design



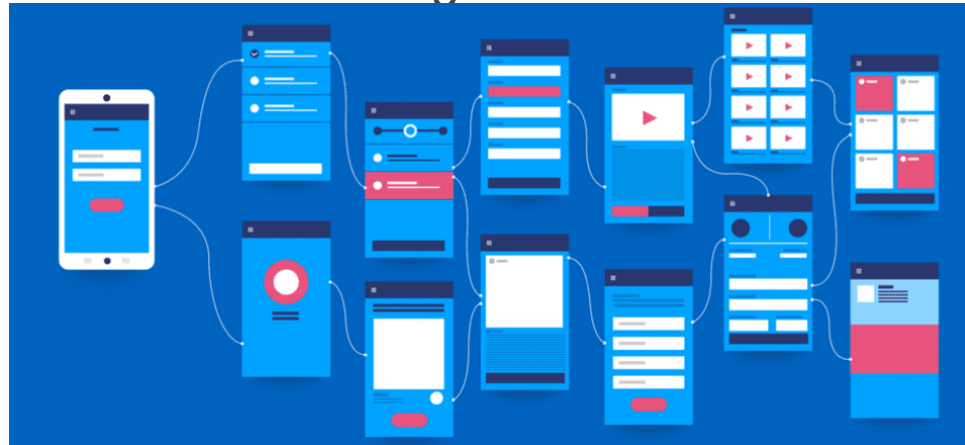
- ✧ User interface is the front-end application view to which user interacts in order to use the software.
- ✧ User can manipulate and control the software as well as hardware by means of user interface.
- ✧ Today, user interface is found at almost every place where digital technology exists, right from computers, mobile phones, cars, music players, airplanes, ships etc.

Software User Interface Design



✧ The software becomes more popular if its user interface is:

- Attractive
- Simple to use
- Responsive in short time
- Clear to understand
- Consistent on all interfacing screens



Software User Interface Design



- ✧ UI is broadly divided into two categories:
 - Command Line Interface
 - Graphical User Interface

Command Line Interface



Command Parameters

Output

Cursor

Command Prompt

```
Pinaca:lib gopal$ ls -al
total 12264
drwxr-xr-x@ 21 gopal  staff    714 Jul  2  2013 .
drwxr-xr-x@ 14 gopal  staff    476 Oct 24  09:20 ..
-rw-r--r--@ 1 gopal  staff  15264 Jul  2  2013 annotations-api.jar
-rw-r--r--@ 1 gopal  staff  54142 Jul  2  2013 catalina-ant.jar
-rw-r--r--@ 1 gopal  staff 134215 Jul  2  2013 catalina-ha.jar
-rw-r--r--@ 1 gopal  staff 257520 Jul  2  2013 catalina-tribes.jar
-rw-r--r--@ 1 gopal  staff 1581311 Jul  2  2013 catalina.jar
-rw-r--r--@ 1 gopal  staff 1801636 Jul  2  2013 ecj-4.2.2.jar
-rw-r--r--@ 1 gopal  staff   46085 Jul  2  2013 el-api.jar
-rw-r--r--@ 1 gopal  staff  123241 Jul  2  2013 jasper-el.jar
-rw-r--r--@ 1 gopal  staff  599428 Jul  2  2013 jasper.jar
-rw-r--r--@ 1 gopal  staff   88690 Jul  2  2013 jsp-api.jar
-rw-r--r--@ 1 gopal  staff  177598 Jul  2  2013 servlet-api.jar
-rw-r--r--@ 1 gopal  staff    6873 Jul  2  2013 tomcat-api.jar
-rw-r--r--@ 1 gopal  staff  796527 Jul  2  2013 tomcat-coyote.jar
-rw-r--r--@ 1 gopal  staff  235411 Jul  2  2013 tomcat-dbcp.jar
-rw-r--r--@ 1 gopal  staff   77364 Jul  2  2013 tomcat-i18n-es.jar
-rw-r--r--@ 1 gopal  staff   48693 Jul  2  2013 tomcat-i18n-fr.jar
-rw-r--r--@ 1 gopal  staff   51678 Jul  2  2013 tomcat-i18n-ja.jar
-rw-r--r--@ 1 gopal  staff  124006 Jul  2  2013 tomcat-jdbc.jar
-rw-r--r--@ 1 gopal  staff   23201 Jul  2  2013 tomcat-util.jar
Pinaca:lib gopal$
```

Graphical User Interface



Software Implementation Challenges



✧ Code-reuse

- Programming interfaces of present-day languages are very sophisticated and are equipped huge library functions.
- Still, to bring the cost down of end product, the organization management prefers to re-use the code, which was created earlier for some other software.
- There are huge issues faced by programmers for compatibility checks and deciding how much code to re-use.

✧ Version Management

- Every time a new software is issued to the customer, developers have to maintain version and configuration related documentation.
- This documentation needs to be highly accurate and available on time.